

Milestone Data Quality

Systems Analysis and Functional Requirements

Version	1.0
Status	Final
Classification	Portfolio Project
Prepared by	N M Ziyaf, Supply Chain Business Analyst
Date	30 June 2026
Domain	Freight Forwarding and Supply Chain

This document is a portfolio project. It analyses a systems level data quality gap in milestone event capture within a CargoWise One freight forwarding environment, and sets out the functional and non functional requirements that would close that gap. The figures quoted are drawn from a companion SQL query library run against a representative shipment dataset.

Executive Summary

I examined how shipment milestone events are recorded inside our CargoWise One freight forwarding environment, and I found a data quality problem that originates in the design of the application rather than in the conduct of the operators who use it. Milestone events such as booking confirmed, departed origin, customs cleared, arrived destination and delivered to consignee are the backbone of every shipment status view, every customer update and every on time performance report. When I measured how reliably those events are captured, the picture was weak in a way that is consistent and structural.

Across the dataset I analysed, 21.9 percent of shipments are missing at least one milestone timestamp, the average gap between when a milestone actually occurred and when it was entered into the system is roughly 2.3 days, and the overall data quality score I calculated sits at 38.8 percent. On the threshold I defined for an acceptable record, 61.2 percent of shipments fail. I also found 162 records that are chronologically impossible, where a later milestone carries an earlier timestamp than an event that must precede it, for example a delivery recorded as happening before customs clearance.

The reason these numbers are so consistent is that the application does not enforce anything. Milestone capture is a manual, free typing activity with no trigger that obliges a milestone to be recorded when the underlying event happens, no validation that the timestamp entered is plausible, and no exception flag when a milestone is late or absent. The system faithfully stores whatever is typed, including nothing at all. This is a system design gap. The same gap would produce the same results regardless of which operator was at the keyboard, which is exactly what the operator level figures show. My recommendation is therefore to add a validation and exception layer to the milestone capture function so that the application enforces the data quality that the business already assumes it has.

Current System Behaviour

This section describes how the milestone capture function behaves today, based on how the application is configured and how the resulting data looks when queried. I have separated the behaviour into how a milestone is captured, what the system validates, and how that shapes the operator workflow, because the gap sits at the boundaries between these three things.

Capture mechanism

Each shipment carries a set of milestone fields, and each milestone holds an expected timestamp and an actual timestamp. The actual timestamp is the moment the business cares about, because it represents when the event genuinely happened. In the current configuration that actual timestamp is populated by hand. An operator opens the shipment record and types or selects the date and time for the milestone. There is no integration trigger that writes the milestone automatically when the event is signalled by a carrier message, a customs status feed or a gate event, and there is no rule that the milestone must be present before the shipment can progress. Because capture depends entirely on a person choosing to enter a value, the actual timestamp is frequently entered well after the event, and is sometimes never entered at all.

Validation rules

The application applies no validation to the milestone timestamp at the point of entry. It does not check that the value falls within a plausible window relative to the event it represents, it does not check that milestones are entered in their natural chronological order, and it does not require that the field be completed. The consequence of the missing order check is the 162 chronologically impossible records I found, where the system accepted a delivered to consignee timestamp that pre dates the customs cleared timestamp on the same shipment. A single plausibility rule would have rejected every one of those at entry. The absence of a mandatory rule is why 21.9 percent of shipments reach reporting with at least one milestone blank, and the absence of a timing window is why the average entry happens about 2.3 days after the fact rather than close to real time.

Operator workflow

Because nothing in the system compels timely or complete capture, the operator workflow absorbs the milestone update as a low priority background task that competes with live operational work. Milestones are often updated in batches when an operator returns to a shipment for another reason, which is the mechanism that produces the multi day entry delay. I want to be clear that this is not an operator discipline failure. The operators are working exactly as the system allows them to work, and 35.3 percent of shipments pass through more than one operator, so no individual owns the complete milestone history of a shipment end to end. When responsibility is shared and the tool enforces nothing, gaps are the predictable outcome, not the exception.

Required System Behaviour

The following statements describe what the application should do once a validation and exception layer is added to milestone capture. They are written as system obligations, because the change I am proposing is a change to the software, not a change to how operators are asked to behave. Each statement targets one of the failure modes identified in the previous section.

1. The system shall capture milestone timestamps automatically from available integration points, including carrier status messages, customs status feeds and gate events, so that capture does not depend on manual entry alone.
2. The system shall require that a milestone timestamp is present before a shipment can advance past the stage that milestone represents, so that milestones cannot be silently skipped.
3. The system shall validate every milestone timestamp against a configurable expected time window and reject or challenge values that fall outside it, so that implausible and late entries are caught at the point of entry.
4. The system shall enforce chronological order across the milestone sequence and prevent a later milestone from being saved with a timestamp earlier than a milestone that must precede it.
5. The system shall raise a structured exception flag whenever a milestone is missing or exceeds its expected window, rather than passing the record to reporting unmarked.
6. The system shall record which operator entered or modified each milestone and when, so that the milestone history is fully auditable.

Functional Requirements

The functional requirements below translate the required behaviour into specific, testable obligations on the application. Priority is expressed as Must, Should or Could. The acceptance criteria describe the condition under which I would consider each requirement satisfied.

ID	Requirement	Priority	Acceptance Criteria
FR-01	Automatically capture milestone timestamps from configured integration points (carrier messages, customs feeds, gate events) and write them to the milestone actual timestamp field.	Must	When a recognised integration event is received, the corresponding milestone actual timestamp is populated within the configured latency without any manual entry.
FR-02	Enforce mandatory completion of each milestone before the shipment can progress beyond that stage.	Must	An attempt to advance a shipment past a stage whose milestone is blank is blocked and the user is informed which milestone is outstanding.
FR-03	Validate each milestone actual timestamp against a configurable expected time window for that milestone type.	Must	A timestamp entered outside the configured window is rejected or routed for confirmation, and no out of window value is stored silently.
FR-04	Enforce chronological consistency across the full milestone sequence on a shipment.	Must	Saving a milestone whose timestamp precedes an earlier required milestone is prevented, and the test dataset produces zero chronologically impossible records.
FR-05	Raise a structured exception flag when a milestone is missing or breaches its expected window, and expose that flag to downstream reporting.	Must	Every missing or out of window milestone carries a queryable exception flag, and reporting can distinguish flagged from clean records.
FR-06	Maintain an audit trail capturing the operator identity, action and timestamp for every milestone entry and modification.	Should	For any milestone, the system returns who entered it, who last changed it, and when each action occurred.
FR-07	Provide a configurable exception queue so that flagged shipments can be reviewed and resolved without scanning every record.	Should	Flagged shipments appear in a filterable queue, and resolving the underlying milestone clears the flag.
FR-08	Allow the expected time windows and tolerance thresholds to be adjusted per milestone type and per trade lane without code change.	Could	An authorised user can change a window or threshold through configuration and the new value takes effect on subsequent

ID	Requirement	Priority	Acceptance Criteria
			validations.

Non Functional Requirements

The non functional requirements describe the qualities the validation and exception layer must exhibit to be trusted in day to day operation. They cover the speed of automated capture, the completeness of the audit record, and the ability to tune the rules as trade conditions change.

ID	Category	Requirement	Target
NFR-01	Performance	Automated milestone capture and validation must complete quickly enough that the milestone reflects near real time status.	Integration sourced milestones are written and validated within five minutes of the event being received.
NFR-02	Performance	Validation at the point of manual entry must not noticeably slow the operator.	Entry time validation returns a result within one second under normal load.
NFR-03	Auditability	Every milestone creation and modification must be permanently attributable and reconstructable.	The audit trail is complete, immutable and retained in line with the records retention policy.
NFR-04	Auditability	Exception flags must be queryable historically so that data quality can be trended over time.	Flag state and history can be reported by month, milestone type, lane and operator.
NFR-05	Configurability	Rules, windows and thresholds must be maintainable by authorised business users.	Changes are applied through configuration without a software release and are themselves audited.

Data Quality Evidence

The figures in this section come from the companion SQL query library, which I ran against a representative shipment dataset built to mirror the milestone structure of the live environment. I am quoting them here so that the requirements are anchored in measured evidence rather than impression. Each figure points back to one of the system gaps already described.

Measure	Result	What it means for the business
Shipments missing at least one milestone	21.9%	More than one shipment in five reaches reporting with an incomplete event history, because completion is not mandatory.
Average entry delay across milestones	~2.3 days	Status is on average more than two days stale, because there is no timing window and no automated capture.
Overall data quality score	38.8%	On a combined completeness, timeliness and validity measure, the milestone data is failing far more often than it is passing.
Shipments failing the record quality threshold	61.2%	Almost two thirds of shipments would not pass a basic acceptance check, so reports built on this data carry that error forward.
Shipments passing through more than one operator	35.3%	Over a third of shipments have shared milestone ownership, which is why no single person can be held to account for completeness.
Chronologically impossible records	162	These records violate the natural order of events and could only exist because no order validation runs at entry.

Read together, these measures describe the same root cause from several angles. Completeness fails because nothing is mandatory, timeliness fails because nothing enforces a window, and validity fails because nothing checks order or plausibility. The impossible records are the clearest proof that the gap is in the application, because no amount of operator care can produce a record that the software should never have accepted in the first place.

Recommendation

My recommendation is to treat this as a system design change and to introduce a validation and exception layer into the milestone capture function, rather than to respond with more manual checking or operator reminders. The evidence shows that the failure is structural and uniform, so the fix has to live where the structure lives, which is in the application. Adding controls to the tool addresses the root cause. Adding effort to the workflow only treats the symptom and would decay the moment attention moved elsewhere.

Concretely, the application should capture milestones automatically wherever an integration point exists, validate every timestamp against a configurable expected window, enforce chronological order so impossible records cannot be saved, and raise a structured exception flag whenever a milestone is missing or late so that reporting can see the difference between trustworthy and untrustworthy data. With those controls in place I would expect the proportion of shipments missing a milestone to fall sharply from the current 21.9 percent, the average entry delay to compress from roughly 2.3 days toward near real time on integration sourced events, and the count of chronologically impossible records to reach zero, because the validation would refuse them at the point of entry. The overall data quality score would rise as a direct consequence of closing each of those gaps.

The change is also proportionate. The exception layer concentrates human attention only on the records that genuinely breach a rule, instead of asking the team to spot check everything, so it improves data quality and reduces manual effort at the same time. That combination is what makes this a system level improvement rather than a temporary correction.

Document Control

This document is a portfolio project prepared to demonstrate systems analysis and requirements definition in a freight forwarding context. The data is drawn from a representative dataset rather than a live production system.

Version	Date	Author	Summary of change
1.0	30 June 2026	N M Ziyaf	Initial issue. Systems analysis of milestone data quality gap, with functional and non functional requirements.

Prepared by N M Ziyaf, Supply Chain Business Analyst. Contact: [linkedin.com/in/ziyafmohamed](https://www.linkedin.com/in/ziyafmohamed)